

OBJECT-ORIENTED (JAVA)

- **Inheritance (IS-A):** Subclass instance can be assigned to superclass ref (Animal a = new Dog();). Compiler uses ref as a “mask,” restricting access to superclass methods only.
- **Type Safety:** Cannot assign superclass to subclass ref (Dog d = new Animal();). Object lacks subclass methods (e.g., bark()); causes crash.
- **Exceptions:** Only objects inheriting from java.lang.Throwable can be thrown. Strings are not Throwable.

UML NOTATION

- **Abstract Classes:** Italicized or stereotyped. Standard text implies non-abstract.
- **Visibility:** Public (+), Private (-).
- **Multiplicity:** Defines instance connections (e.g., 1 . . . *, 0,1).
- **Association Class:** Connects to an association relationship to add attributes/operations (e.g., Payment class for a User-Shop “Pays” relationship).
- Underlined (:Container): Represents an object (an instance of a class).
- Not underlined (:Container or Container): static class

METHODOLOGY & ARCH

- **Breadth-First (Iterative):** Evolves all major components in parallel (thin-slice).
- **Depth-First (Incremental):** Fully implements specific features one by one.
- **Waterfall:** Rigid, sequential (Req → Design → Code → Test). Best for stable reqs. In a pure Waterfall model, you do not release working software in phases but in one shot after gathering all requirements.
- **n-Tier/Layered:** Divides app into horizontal layers (Pres, Biz, Data) to isolate responsibilities.
- **MVC:** Separation of Concerns via Data (Model), UI (View), Input (Controller).
- **CI/CD:** CI automates merge/test; CD automates release. (They are not the same).
- **User Story:** “As a [Role], I want [Func] so that [Benefit].”
- **Non-Functional Req:** e.g., “Low Latency for Order Synchronization.”
- Breadth-First Development (BFD): or a “horizontal” approach, focuses on implementing all major components and architectural layers of a system in parallel

DESIGN PATTERNS

- **Singleton:** Class has only 1 instance. Acts globally, increases coupling, hard to replace with stubs in testing.
- **Facade:** Simplified interface to complex internal classes. Hides internals (e.g., UI accesses Logic component via Facade without knowing it contains a Book class).

DESIGN PRINCIPLES

- **SRP:** Single Responsibility. 1 reason to change.
- **OCP:** Open for extension, Closed for modification. Fixes poor extensibility,

- **LSP:** Liskov Sub. Derived classes must substitute base classes. Violations: Subclass throwing new error, or reducing capability.
- **ISP:** Interface Segregation. Don’t force clients to depend on unused methods.
- **DIP:** Dependency Inversion. High & low modules depend on abstractions, not details.
- **Law of Demeter (LoD):** Least knowledge. Object only calls methods of itself, params, created objs, or direct fields (immediate friends).
- **DRY:** Don’t Repeat Yourself. Single authoritative representation.
- **YAGNI:** You Aren’t Gonna Need It. Don’t code for hypothetical futures.

TESTING & QA

- **System vs Acceptance:** System checks tech specs/edge cases (heavy pos/neg testing). Acceptance (by client) validates biz reqs (happy-paths).
- **Dynamic Analysis:** Requires executing code (e.g., code coverage tools).
- **Static Analysis:** Inspects code without running it.
- **Equivalence Partitioning:** Group inputs behaving similarly. Test boundaries (0, 1, MAX). One issue per partition. NULL is an option.
- **Defensive Prog:** Runtime checks (null-checks) to prevent crashes (adds slight execution time).
- **Positive Test:** Valid inputs → Success.
- **Negative Test:** Invalid inputs → Throws exception (e.g., quantity=0).
- **Stub:** Simple hardcoded implementation mimicking responses for testing. Replaces complex components.
- **Tools: JUnit** = Testing framework. **Gradle** = Build tool (compile/test/pack). **Jenkins/GH Actions** = CI platforms that trigger tools.

CLEAN CODE & STANDARDS

- **lowerCamelCase:** Locals, Params, Fields.
- **UPPER_SNAKE_CASE:** Constants.
- **Booleans:** Prefix with is, has, can.
- **Issues:** Magic Strings/Nums (use constants), Indentation, Input validation, Missing SRP.
- **Comments:** Explain “Why,” not “What”. Javadocs should start with “Returns the text...”

GLOSSARY & MISC

- **Abstraction:** Suppress complex details below level of interest.
- **Actor:** User role (human/system) outside system.
- **Architecture:** High-level structures/relations.
- **Brainstorming:** Generate diverse/creative ideas.
- **Brooks’ Law:** Adding people to late project makes it later.
- **CLI App:** Command Line Interface application.
- **Coupling:** Degree of interdependence between modules.
- **Domain Expert:** Discipline expert (e.g.,

- Accountant for accounting app).
- **Dynamic Binding:** Late binding; methods resolved at runtime.
- **Static Binding:** Early binding; resolved at compile time.
- **Enterprise App:** High demands (scale, perf, sec).
- **Exception:** Event disrupting normal flow.
- **Feature List:** Grouped by aspect, priority, delivery.
- **Focus Group:** Informal interview in interactive group.
- **Glossary:** Ensures common stakeholder understanding of terms.
- **Pair Programming:** Driver writes code, Observer/Navigator reviews. Switch frequently.
- **Polymorphism:** Diff objects respond own way to identical messages.
- **Prototype:** Mockup to get feedback, validate tech (proof-of-concept), or field-test.
- **Type Signature:** Parameter type sequence (order matters). Return type/param names are **not** part of it.
- **Use Case:** Sequence of actions yielding observable result. Contains Main flow & Extensions (e.g., 3a. Camera blocked → notify → user unblocks → resume).

Git / VCS

- **Commit:** (v) saving change to history, (n) a revision.
- **Stage:** Instruct Git to prep file for commit.
- **RCS:** Revision Control Software automates tracking.
- **Repo:** DB of history tracked by RCS.
- **Working Dir:** Root directory revision-controlled by Git.

1. QA & TESTING

QA = Validation + Verification

- **Validation:** Building the *right system?* (Reqs correct?)
- **Verification:** Building the *system right?* (Implemented correctly?)

Testing Levels

- **Unit Testing:** Test individual units (methods/classes).
- **Integration Testing:** Test if parts *work together*. Focuses on interactions (e.g., use stubs for Engine/Wheel, then test real pieces integrated).
- **System Testing:** Test whole system against the *system specification*. Focuses on external behavior. Done by QA team. Tests Non-Functional Reqs (NFRs) like load/security, and graceful failure beyond limits.
- **Acceptance (UAT):** Ensure it meets *user reqs*. Done by user/customer team on deployment site.

Testing Types & Approaches

- **Alpha:** Users test in controlled developer env.
- **Beta:** Subset of target users test in natural setting.
- **Dogfooding:** Creators use own product.
- **Scripted:** Pre-written systematic test cases. High coverage but slow.

- **Exploratory:** Ad-hoc, on-the-fly. Fast but might miss edge cases. (Doing both is ideal).
- **Regression:** Re-testing to detect unintended effects after mods. Best automated.
 - **TestFX:** Automates JavaFX UI.
 - **Visual Studio:** Record-replay GUI automation.
 - **Selenium:** Automates web app UIs.
- **Test-Driven Dev (TDD):** Write tests *before* SUT (System Under Test). Cycle: Red (fail) → Green (pass) → Refactor. Evolves in increments.

Coverage & Strategies

- **Test Coverage:** Metric for code exercised by tests.
- **Function/Method:** % of functions executed.
- **Statement:** % of LOC executed.
- **Decision/Branch:** If-statements evaluate to both true/false (tests actual control flow branches).
- **Condition:** Every boolean sub-expression evaluated true/false. (Not same as decision).
- **Combinatorial Strategies:**
 - **All combinations:** Every unique combo of inputs.
 - **At least once:** Each test input used ≥ 1 times. Heuristic: Valid inputs ≥ 1 time in a positive test; test invalid inputs individually before combining.
 - **All pairs:** All combos for any given *pair* of inputs are tested. Optimizes coverage vs execution time.

Test Data Selection: Equivalence partitions (grouping inputs with same behavior) & Boundary values (testing edges).

Formal Verification: Uses mathematical techniques to prove correctness. Highly specialized/expense. Used in safety-critical software (flight control).

2. CODE QUALITY & MAINT

Code Review & Analysis

Code Review: Systematic exam to improve code.

- PRs, Pair Programming, Formal Inspections.

Analysis Tools:

- **Static Analysis:** Examines unexecuted code (unused vars, style errors). IDEs do this live. Java tools: CheckStyle, PMD, FindBugs.
- **Linters:** Subset of static analyzers for “cleaner” code.
- **Dynamic Analysis:** Requires execution. Gathers performance/memory stats.

Debugger Tool: Pauses execution, step-through line-by-line, inspect internal state. *Recommended bug-fixing approach.*

Refactoring

- Restructuring code in small steps without modifying external behavior. First write working code, then improve quality.
- **NOT Rewrite/Bug-fix:** Does not alter external behavior or discard entire codebases.
 - **Benefits:** Hidden bugs become visible, simpler code runs faster/easier for compiler

- to optimize.
 - **Examples from Catalogs:** Consolidate Duplicate Conditional Fragments (move shared code outside ‘if’), Remove double negative, Replace magic literal, Replace param with explicit methods.
- Documentation**
- **Dev-as-user:** How to reuse your component.
 - **Dev-as-maintainer:** Explaining complex internals.
 - **Four types:** Tutorials, How-to guides, Explanations, References.
 - **JavaDoc:** HTML API generation. Classes: “Represents...” . Methods: “Returns...” or “Performs...” . Explain all params, returns, and throws.

3. REUSE & INFRASTRUCTURE

Reuse risks: Overkill (sledgehammer for a nut/bloat), lack of maturity (risk of dying off), restrictive licenses, bugs/vulnerabilities, malicious compromised dependencies.

Mechanisms

- **Library:** Modular, general code used “as is”.
- **API:** The contract/interface. For a class, it’s the collection of public methods to invoke.
- **Framework:** Compartmentalized similarity. Implements default behavior. Facilitates adaptation/customization. Meant to be extended (e.g., Eclipse plugins, JUnit, JavaFX, Tkinter). Uses Inversion of Control.
- **Platform:** Provides a *runtime environment*. Can bundle tools/libs. (e.g., JavaEE, .NET, OS).

4. REQUIREMENTS

- **Brownfield:** Replace/update existing system.
- **Greenfield:** Totally new system from scratch.
- **Stakeholder:** Anyone involved/affected (users, sponsors, devs, gov, interest groups).

Types of Reqs & Specs

- **Functional (FR):** What system should do.
- **Non-Functional (NFR):** Constraints on dev/ops. Includes: Business rules, constraints (budget/schedule), Tech (32/64-bit), Perf (resp time), Quality (novice usability), Process, Project Scope, Noteworthy points.

Reqs Spec vs System Spec:

- **Reqs:** Normal conditions; terms of problems to solve; interface for intended end-users.
- **System:** Includes graceful failures; terms of how system solves problems; internal APIs for devs/testers.

Elicitation & Use Cases

- Gathering methods:** Brainstorming, Surveys, Observation, Interview, Focus groups, Prototyping (validates tech & gets feedback), Product surveys (study shortcomings).
- Use Cases (UC):** Interaction between Actor and System for specific functionality. Captures FRs.
- **Step:** Gives actor’s intention.
 - **MSS (Main Success Scenario):** Standard flow.

- **Extensions:** Add-ons to MSS for alternative/exceptional flows (e.g., 3a happens after step 3; *a happens anywhere).
 - **Include:** UC can include another UC (underlined).
 - **Preconditions:** Required state before UC starts.
 - **Guarantees:** Promised state after UC ends.
 - **For Testing:** MSS is one test case, variations are others. UC lacks exact data, so tester applies equivalence partitions.
- Feature List:** Features grouped by aspect/priority.

5. DESIGN & ARCHITECTURE

- **Product/External Design:** User behavior. By UX/BAs.
- **Implementation/Internal Design:** Implementation tech. By Architects/Devs.
- **Approaches:** Top-down (high-level first, stable architecture), Bottom-up (components first, good for variations/repurposing), Mixed.

Core Principles

Abstraction: Suppressing lower-level complex details.

- **Data Abstraction:** Thinking in bigger entities.
- **Control Abstraction:** Focusing on higher-level flow.
- **Coupling & Cohesion:**
- **Coupling:** Measure of dependence. *Low is good*. Strong coupling: change in X *potentially* requires change in Y.
- **Cohesion:** Measure of related/focused responsibilities. *High is good*. Keeps related tasks together, unrelated out.

Patterns & Architectures

Design Pattern Format:

- **Context:** Scenario/situation.
- **Problem:** Main difficulty to resolve.
- **Solution:** Core general details (requires refinement).
- **Anti-patterns:** Common wrong/inferior solutions.
- **Consequences:** Pros/Cons.
- **Architectural Styles:** (Structure of specific system category, high scale reuse).
- **Client-Server:** 1+ server, 1+ clients (Distributed).
- **Transaction Processing:** Workload divided into transactions handled by a *dispatcher* (queuing, undo).
- **Service-Oriented (SOA):** Accessible services combined via centralized Enterprise Service Bus (ESB). E.g., XML web services. (NOTE: NOT microservices).
- **Event-Driven:** Detects events from *emitters*, sends to *consumers*. Flow controlled by events. Common in GUI.

6. PROCESS & MANAGEMENT

SDLC Models

- **Sequential (Waterfall):** Linear progression through stages. Hard to pivot.
- **Iterative:** Goes through SDLC in iterations. Each iteration builds new version based on

feedback.

Agile Manifesto Principles:

- Individuals/interactions > processes/tools.
- Working software > comprehensive docs.
- Customer collab > contract negotiation.
- Responding to change > following a plan.

Extreme Programming (XP): Customer focus, teamwork, simplicity, feedback. Small repeated releases.

Scrum Framework:

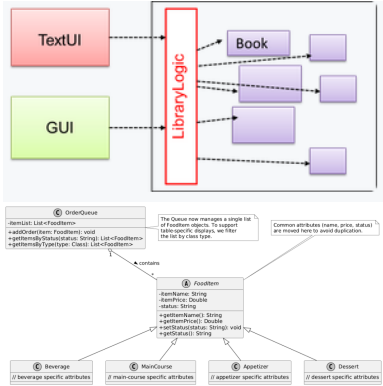
- **Scrum Master:** Maintains processes (pseudo-PM).
- **Product Owner:** Reps stakeholders/business.
- **Team:** Cross-functional execution (dev, test, design).
- **Sprints:** Iterations (1 week to 1 month). Yields potentially deliverable product increment.
- **Daily Scrum:** Key syncing practice.

Project Management Tracking

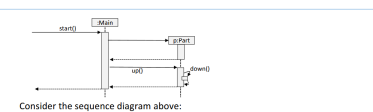
- **WBS (Work Breakdown Structure):** Divides total work into clear, well-defined units/subtasks. Includes prerequisites and effort estimates.
- **Estimation:** Measured in man hour/day/month. Tasks must define exactly when they are "done".
- **Milestone:** End of stage indicating significant progress. Takes dependencies/priorities into account.
- **Buffer:** Time set aside for unforeseen delays.
- **Gantt Chart:** 2D bar-chart (Time vs Tasks horizontal bars). Tracks who/ongoing/done.
- **PERT Chart:** Graphical directed graph. Nodes = effort/tasks, Arrows = precedence/sequence.

Team Structures

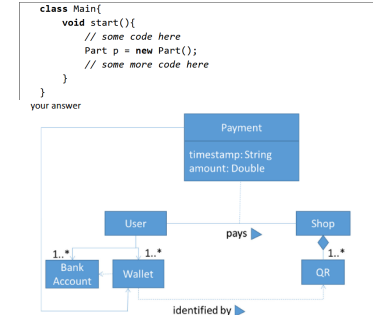
- **Egoless:** Every member equal in responsibility.
- **Chief Programmer:** Single authoritative figure directing.
- **Strict Hierarchy:** Tree system of strict organization.



19. Method overloading can happen within a single class.



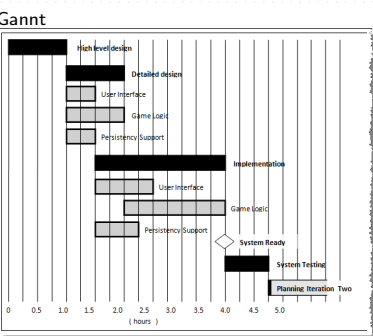
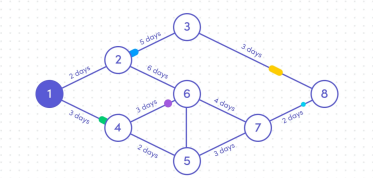
20. The code below is compliant with the diagram



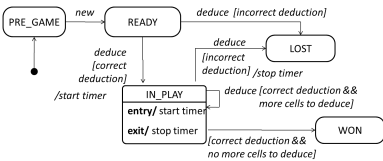
extreme



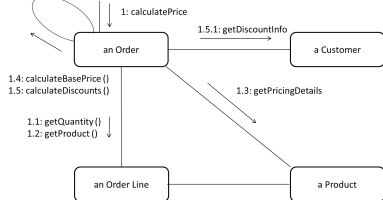
PERT



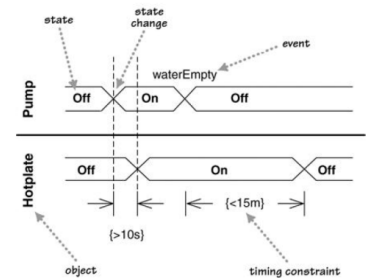
State



Communication



Timing



MISC

- Defining APIs (Application Programming Interfaces) or "contracts" between components early in the design phase is a highly recommended best practice. It allows different teams (e.g., frontend and backend) to work in parallel
- While highly recommended for most projects, an architecture diagram is not strictly mandatory.
- You should only use design patterns when they solve a specific problem you are facing.
- While consulting customers is vital during the requirements gathering and user experience phases, the word "always" makes this tricky.
- ALT vs OPT ==> 'if-else' or 'switch' VS 'if' without else
- For the UML 'loop's condition, if it is just a count (v10), write '10 times' rather than a condition
- directed association is a subset of association
- indentation is 2 spaces instead of 4
- 'git fetch' does not automatically merge changes unlike git pull. pull = fetch + merge
- In UML, generalization includes abstract classes.
- Security/Authentication: The listing module must require active session token authentication to ensure that only verified, logged-in users can add a bike to the system's database. (Relates to your "secure login" concept)
- Performance/Latency: The system must update the central database and display the newly listed bike in search results within 2

seconds of the owner submitting the listing. (Relates to your "live updates" concept)

- Both branch nodes and merge nodes in activity diagram are diamond shapes. Guard conditions must be in square brackets.

Dependency

- **Visual Style:** A - - > B
- **Meaning (A to B):** A uses B temporarily.
- **Additional Details:** This is the weakest relationship. It means A uses B in some way (e.g., B is a parameter in a method of A, or B is a local variable inside a method of A). If B changes, A might need to be updated, but A does not "hold" B as a permanent field.

Association

- **Visual Style:** A - B
- **Meaning (A to B):** A and B are linked (bidirectional).
- **Additional Details:** A structural relationship where both classes are aware of each other. They can communicate bidirectionally. Usually, this means both classes have a member variable (field) that references the other.

Directed Assoc.

- **Visual Style:** A -> B
- **Meaning (A to B):** A knows B (unidirectional).
- **Additional Details:** This defines "Navigability." Class A has a field/attribute of type B, but Class B does not have a reference to Class A. A can send messages to B, but not vice versa.

Generalization

- **Visual Style:** A -> B
- **Meaning (A to B):** A is a type of B (Inheritance).
- **Additional Details:** A is a subclass (child) and B is the superclass (parent). A inherits all the attributes and methods of B. B is the more general concept, and A is the more specialized version.

Realization

- **Visual Style:** A - - > B
- **Meaning (A to B):** A implements Interface B.
- **Additional Details:** B is typically an Interface or an Abstract Class. A provides the actual code (the "realization") for the methods defined in B.

Aggregation

- **Visual Style:** A o-- B
- **Meaning (A to B):** A contains B (B can survive alone).
- **Additional Details:**
- **Composition**
- **Visual Style:** A - B
- **Meaning (A to B):** A owns B (B dies with A).
- **Additional Details:**